Design Document : Neural Accelerator for RISCV

Abstract:

The Neural and ML Accelerator is a custom ASIC accelerator designed for low-power, high-throughput convolutional neural network (CNN) inference at the edge. It features a 32-bit RISC-V management core, a systolic array for matrix multiplication, and dedicated on-chip memory to minimize off-chip DRAM traffic.

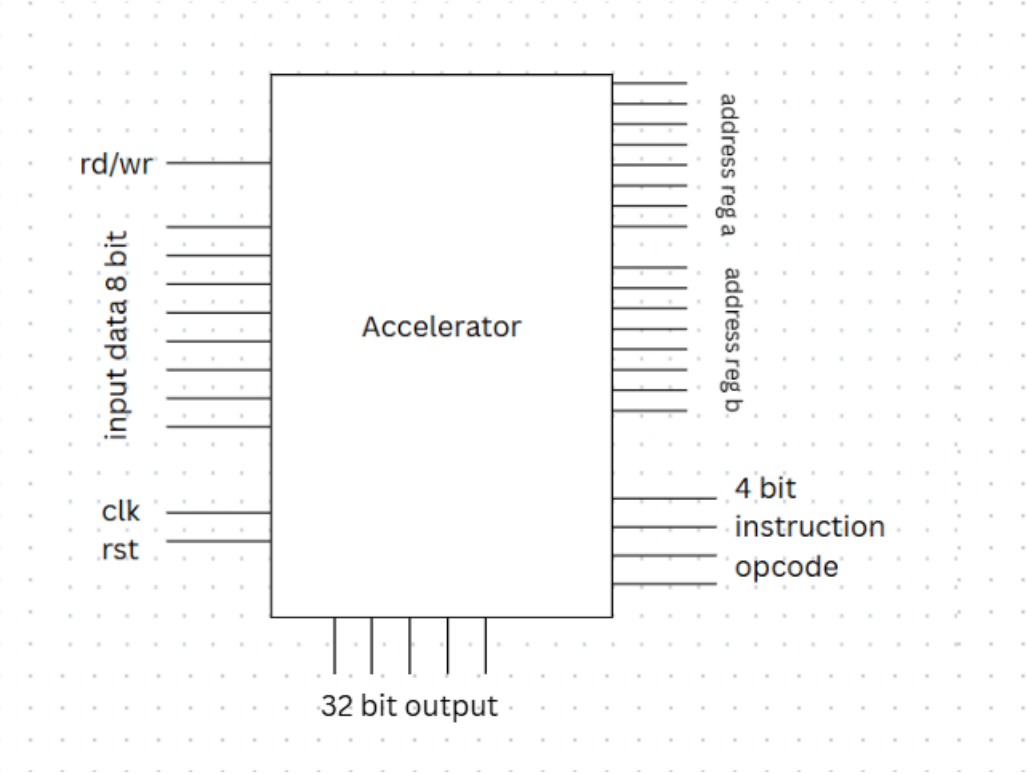
In addition to the matrix acceleration engine, the architecture integrates multiple hardware modules commonly required during neural network forward-pass processing, such as activation, pooling, normalization, accumulation units. The accelerator follows a modular instruction-set-based architecture and pipeline inspired by the principles of RISC-V, where different compute modules are selectively multiplexed and controlled through configurable instructions.

As a dedicated edge-processing accelerator chip, the design is optimized to execute machine learning workloads with significantly lower latency and power consumption compared to general-purpose processors. By performing computations locally on-device, the accelerator enables faster real-time inference for applications such as computer vision, autonomous systems, IoT devices, robotics, and intelligent sensing without relying heavily on cloud connectivity.

Key Features

- 32-bit RISC-V inspired instruction architecture
- Modular opcode-driven execution engine
- 3-stage pipelined datapath
- Dedicated 8×8 systolic array
- INT8 computation with 32-bit accumulation
- Edge-AI optimized low-power architecture
- Support for neural-network activation and quantization modules

Architecture:



1. **Instruction Architecture**

The instruction format contains:

Field	**Purpose**
rd/wr	Read or write operation
addr_A	Base address for Matrix A
addr_B	Base address for Matrix B
opcode	Select operation
clk	Clock signal
rst	Reset signal
data_in	Input data during write mode

Control and Interface Signals

The accelerator is controlled using a simple instruction-based interface containing the following signals:

- **rd/wr bit**
    - Controls load/store operation
    - 0 → Read/Execute mode
    - 1 → Write mode
- **8-bit Register A Address (addr_A)**
    - Selects starting address of Matrix A registers
- **8-bit Register B Address (addr_B)**
    - Selects starting address of Matrix B registers
- **4-bit Instruction Opcode**
    - Supports **16 unique instructions**
    - Selects the operation module to execute
- **clk signal**
    - System clock for synchronous operation
- **rst signal**
    - Resets registers and internal states
- **8-bit Data Input (data_in)**
    - Used during write/load operation to store values into registers
- **32-bit Output (cout)**
    - Outputs final computation result from accumulator registers

Working Principle

- The **opcode** selects the required compute module
- A multiplexer routes inputs and outputs to the selected execution block
- Only the required hardware module becomes active for execution

---

2. **Memory Architecture : Register Bank Structure**

Section	Number of Registers	Width
Register Bank A	16	8-bit
Register Bank B	16	8-bit
Internal Matrix A	64	8-bit
Internal Matrix B	64	8-bit
Accumulator Array	64	32-bit

Register Bank A :Stores a **4×4 Matrix A**

Register Bank B : Stores a **4×4 Matrix B**

The matrices are formed using **consecutive register locations**.

Example:

- If addr_A = X
    - Matrix element A[0][0] → Register X
    - A[0][1] → Register X+1
    - A[0][2] → Register X+2
    - and so on...

The same method is used for Matrix B.

This matrix construction happens during the **Instruction Decode Stage**.

Internal Computation Registers

Padded Matrix Registers

This is a new approach to build a padded matrix to reduce the design complexity by constructing a padded matrix for better timing synchronization.

For internal processing:

- Two separate **8×8 register arrays** are used

- Each register is **8 bits**
- Used for:
    - Matrix padding
    - Parallel computation
    - Alignment with systolic processing structure

These internal arrays temporarily store processed versions of Matrix A and Matrix B before execution.

8×8 Accumulator Array

- Stores intermediate multiplication and accumulation results
- Each accumulator register is **32 bits wide**
- Prevents overflow during large computations

The final **4×4 result matrix** is connected to the cout output line through wired connections from the accumulator registers.

Microarchitecture ,Datapath ,Control Unit:

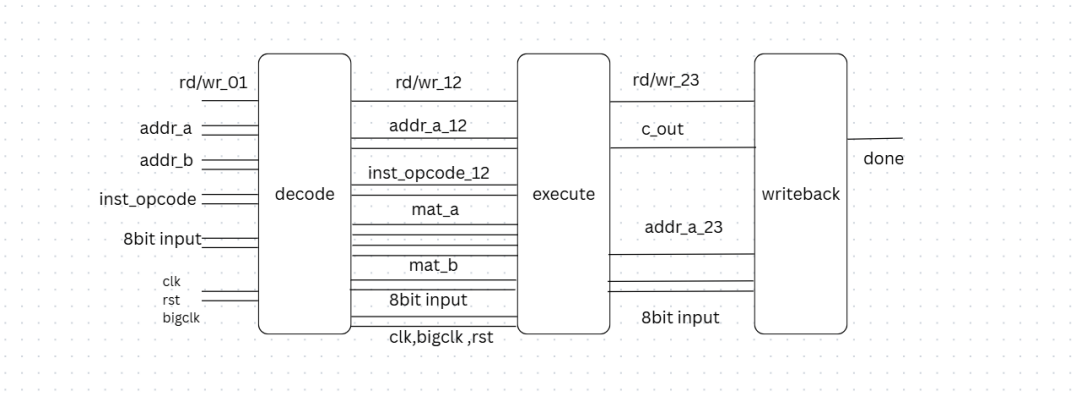
The proposed Neural and ML Accelerator follows a modular microarchitecture consisting of a control unit, Datapath, execution engine, register banks, and a 3-stage pipelined processing structure. The architecture is designed for efficient neural-network feedforward computation and systolic-array-based parallel execution.

Overall FSM Control

This FSM enables synchronization between the processor and accelerator while providing execution status monitoring. The accelerator operates using a finite state machine (FSM) containing three primary states:

- **IDLE** : Default waiting state , Waits for instruction and control signals
- **COMPUTE :** Busy flag is asserted (busy = 1), Instruction execution and systolic computation occur in this stage
- **DONE** : Indicates completion of execution, Processor can read the output/result status from the accelerator

3-Stage Pipeline Architecture



The Datapath is organized into a 3-stage pipeline:

Stage 1 — Data Decode Stage

In the decode stage, register-bank access is performed.

Operations performed in this stage include:

- Reading Register Bank A and Register Bank B
- Constructing matrices using consecutive memory locations

For matrix operations:

- mem[addr_A] → A[0][0]
- mem[addr_A + 1] → A[0][1]
- and similarly for the remaining matrix elements

The same mechanism is used for Matrix B.

For operations requiring only a single register operand:

- Only A[0][0] or the first required register value is taken in the execution stage

For array-based operations:

- The entire row/vector (A[0]) is extracted in the execution stage

The decode stage also forwards opcode ,matrix data ,addresses ,control signals ,clock/reset signals to the execution pipeline stage.

Stage 2 — Execute Stage

The execute stage contains the main computation engine and instruction execution modules. The instruction opcode is routed through a multiplexer structure implemented using the case construct in Verilog. Depending on the opcode, the corresponding execution module becomes active. The architecture currently supports 16 dedicated instruction modules for:

- matrix operations
- neural-network processing operations
- systolic-array-based execution

Two clock domains are used:

- **clk** : Used for systolic data movement and PE shifting operations
- **bigclk** : Used for pipeline-level instruction execution, Generated internally using a counter

Since the systolic array requires 8 clock cycles to complete computation, bigclk is generated using: (bigclk = ~bigclk) after every 8 clk pulses through counter-based clock division logic. This allows synchronization between systolic computation latency and instruction execution timing.

Stage 3 — Write-Back Stage

The final stage performs memory update and output write-back operations. Functions of this stage include:

- Writing c_out back into: register banks ,intermediate registers
- Handling external write operations :

  If the write signal is enabled: mem[addr_A] = input_data; can be performed directly to store external data into the register bank.

The write-back stage also generates: completion signal ,done status ,output forwarding

for processor-level synchronization.

Datapath Structure

The datapath consists of:

- Register Bank A
- Register Bank B
- Matrix construction logic
- Instruction decoder
- Multiplexer-based execution selector
- 16 execution modules
- Systolic computation array
- Accumulator array
- Write-back routing logic

The datapath is optimized for:

- parallel execution
- low-latency matrix computation
- reduced memory access overhead
- efficient edge-AI inference

Current Implementation Status

Currently:

- The execution block and instruction modules are fully implemented
- Decode logic, FSM control, and write-back stages are architecturally defined and partially ideated

The modular pipeline-based design allows scalability and future integration of additional ML operations while maintaining compatibility with the instruction-driven accelerator architecture inspired by RISC-V principles.

Execution block:

alu.v	5/10/2026 1:50 PM	V File	1 KB
bnn.v	5/11/2026 7:09 PM	V File	1 KB
clip.v	6/25/2025 8:41 PM	V File	1 KB
dot_prod.v	5/10/2026 5:54 PM	V File	1 KB
execute_top.v	5/11/2026 7:32 PM	V File	3 KB
mac.v	6/21/2025 11:55 AM	V File	1 KB
matrix_mul.v	5/10/2026 5:41 PM	V File	4 KB
max.v	6/21/2025 1:08 PM	V File	1 KB
ml_accel_fsm.v	5/11/2026 1:04 PM	V File	2 KB
ml_accel_fsm_tb.v	7/16/2025 8:03 AM	V File	2 KB
pe.v	5/11/2026 1:37 PM	V File	2 KB
pipeline.jpeg	5/11/2026 7:32 PM	JPEG File	125 KB
popcount.v	5/11/2026 7:05 PM	V File	1 KB
ReLU.v	6/21/2025 1:15 PM	V File	1 KB
scaling.v	5/11/2026 7:08 PM	V File	1 KB
sigmoid.v	5/11/2026 7:08 PM	V File	1 KB
systolic_array.v	5/12/2026 10:07 AM	V File	5 KB
tanh.v	5/11/2026 7:08 PM	V File	1 KB
text	6/21/2025 11:45 AM	File	1 KB

The execution block forms the main computation engine of the accelerator and is responsible for performing neural-network and machine-learning operations selected through the instruction opcode. The module follows a modular execution architecture where each operation is implemented as an independent hardware block and selected using opcode-based multiplexing. The execute stage operates as the second stage of the 3-stage pipeline and receives decoded data, matrix inputs, control signals, and register operands from the decode stage.

The execution block uses two clock domains. Since the systolic array requires 8 clock cycles for computation completion, bigclk is generated internally using a counter-driven clock division mechanism.

Inputs to Execute Block

- reg_a and reg_b for scalar operations
- matrix_a and matrix_b for matrix operations
- opcode for instruction selection
- addr_a and addr_b for pipeline forwarding
- clk, bigclk, and rst control signals

Outputs

- cout : 32-bit execution result
- Forwarded pipeline control signals:
    - addr_a_next
    - addr_b_next
    - rd_wr_bar_next

Execution Modules

The execution engine currently contains multiple dedicated hardware modules commonly required during neural-network forward-pass computation. Single register input modules are:

ReLU Module

- Implements Rectified Linear Unit activation
- Outputs zero for negative inputs

MAC Module

- Performs signed INT8 multiply-and-accumulate operation
- Used for neural-network accumulation and convolution computation

Dot Product Module

- Computes vector dot product using parallel MAC units
- Used in matrix multiplication and feedforward operations

Popcount Module

- Counts number of active bits in an 8-bit input

- Useful in binary neural network computation

Clip Module

- Restricts outputs within programmable minimum and maximum ranges
- Used for quantized inference control

Scaling Module

- Performs fixed-point scaling and shift operations
- Used during quantization and normalization

Sigmoid Approximation Module

- Implements low-cost sigmoid activation approximation
- Optimized for hardware-efficient inference

Tanh Approximation Module

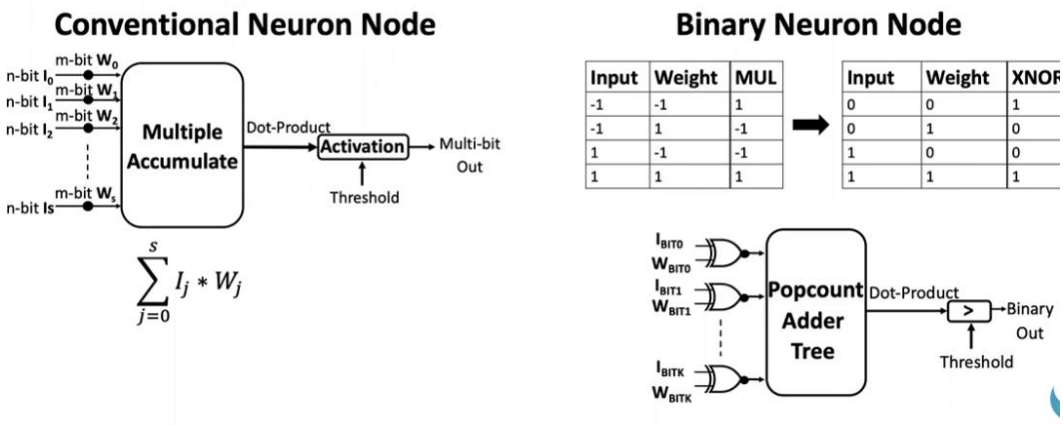
- Provides approximate hyperbolic tangent activation function

Max Finder Module

- Finds maximum value and corresponding index from multiple inputs
- Useful for classification and pooling operations

BNN Module

- Binarized Neural Networks (BNNs) are highly efficient neural networks where weights and activations are constrained to binary values, typically -1 or +1. By replacing standard 32-bit floating-point operations with XNOR and bit-count operations, BNNs drastically reduce memory usage and increase inference speed, making them ideal for edge devices and IoT applications



Matrix Multiplication Module

The accelerator includes a dedicated 4×4 matrix multiplication module designed for neural-network and feedforward computations. The module accepts two signed INT8 matrices and generates a 32-bit output matrix. It is controlled using clk, rst, valid, and done signals. Input matrices are latched and Matrix B is transposed for efficient access. Parallel dot_product_4 modules compute matrix elements simultaneously. Results are written into the output matrix and the done signal is asserted.

Dot Product and MAC Modules

The dot_product_4 module computes vector dot products using four parallel int8_mac units. Each MAC unit performs signed INT8 multiplication and accumulation to generate 32-bit outputs. These modules form the basic computation blocks for matrix multiplication and neural-network arithmetic operations.

Systolic Array Module

The accelerator contains an 8×8 systolic array optimized for parallel matrix computation. Internally, 4×4 input matrices are padded into 8×8 arrays for synchronized systolic data movement and timing alignment.

The systolic array consists of:

- Padding logic (a_pad, b_pad)
- Processing Element (PE) grid
- 8×8 accumulator array
- Dataflow routing network – Facilitated by the PE logic

Processing Element (PE)

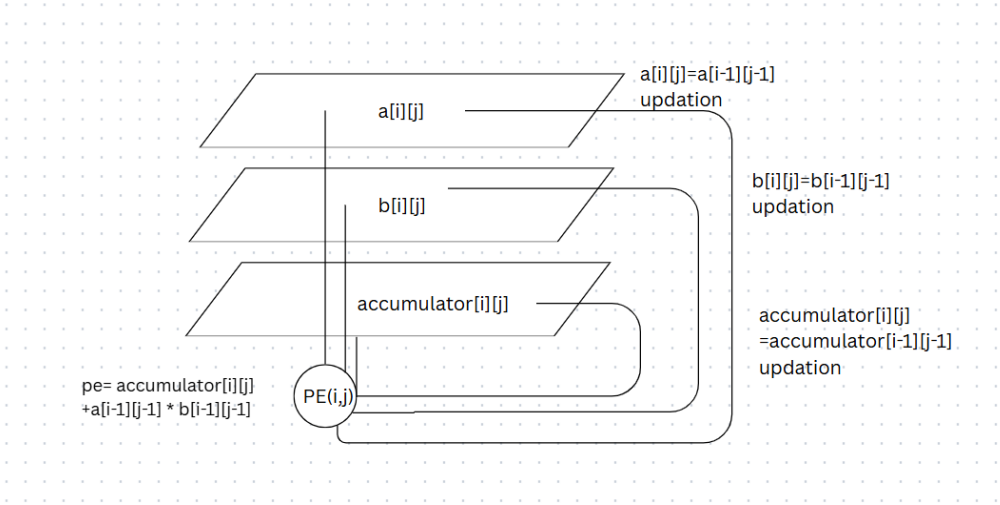
The Processing Element is the core compute unit of the systolic array.

Each PE performs:

- Input data latching
- Multiply-and-Accumulate (MAC) operation
- Horizontal and vertical data forwarding

Partial sums are stored in accumulator registers and propagated through the array during systolic execution.

Accumulator Array : An internal 8×8 accumulator array stores intermediate 32-bit partial sums generated during matrix computation. The final active 4×4 results are forwarded to the output matrix c_out.



Opcode-Based Execution

Instruction execution is controlled using a case-based opcode selection structure implemented in Verilog. Based on the 4-bit opcode, the corresponding execution module is activated and its output is routed to the final 32-bit output register (cout). This modular architecture enables scalability and easy integration of additional ML operations in future versions of the accelerator.

Potential Applications

- Edge AI devices
- Computer vision systems
- Smart surveillance

- Robotics and autonomous systems
- IoT-based intelligent sensing
- Wearable AI devices
- Embedded neural-network inference
- Real-time signal and image processing

The systolic-array-based architecture enables efficient matrix computation while minimizing memory-access overhead, making the design suitable for low-power ASIC implementations.